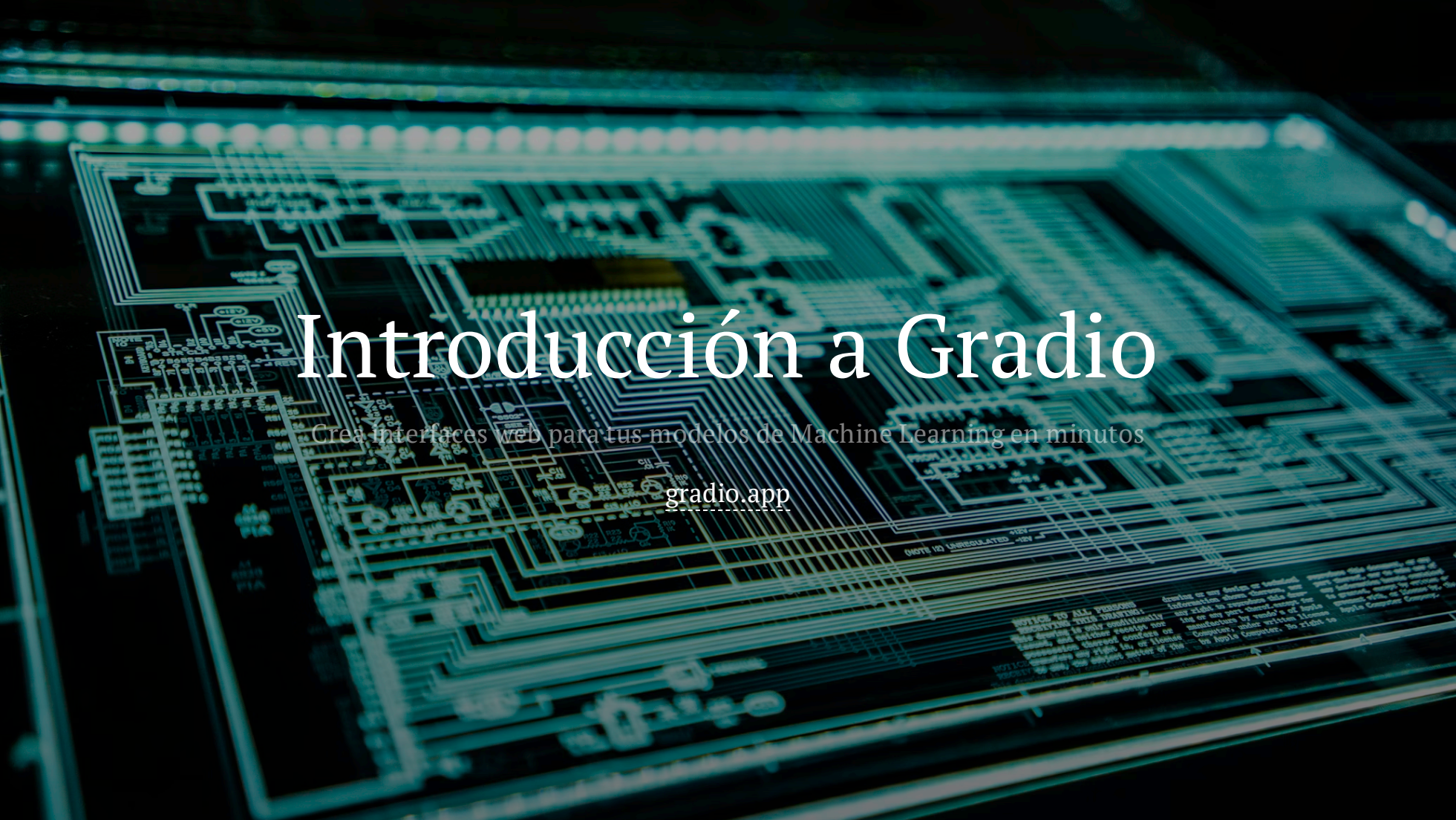




Introducción a Gradio

Crea interfaces web para tus modelos de Machine Learning en minutos

gradio.app

¿Qué es Gradio?

Gradio es la forma más rápida de presentar tu modelo de Machine Learning con una interfaz web amigable para que cualquiera pueda usarlo en cualquier lugar.

- **Rápido:** Define interfaces en pocas líneas de código.
- **Flexible:** Soporta texto, imágenes, audio, video y más.
- **Fácil de compartir:** Genera enlaces públicos automáticamente.
- **Integración:** Funciona perfectamente con Python y las librerías de ML más populares.

Instalación

Para instalar Gradio, simplemente usa `pip` en tu terminal:

```
1  pip install gradio
```

O si usas un entorno específico dentro de un notebook:

```
1  %pip install gradio
```

Hola Mundo con Gradio

El concepto principal es la clase `Interface`, que requiere una función, entradas y salidas.

```
1  import gradio as gr
2
3  def saludo(nombre):
4      return f"¡Hola {nombre}! Bienvenido a Gradio."
5
6  demo = gr.Interface(
7      fn=saludo,
8      inputs="text",
9      outputs="text"
10 )
11
12 demo.launch()
```

Elementos de Interfaz (Inputs/Outputs)

Gradio ofrece múltiples componentes predefinidos para interactuar con tus funciones.

```
1  import gradio as gr
2
3  def procesar_datos(texto, numero, imagen):
4      # Lógica de procesamiento aquí
5      return f"Procesado: {texto}", numero * 2, imagen
6
7  demo = gr.Interface(
8      fn=procesar_datos,
9      inputs=[
10         gr.Textbox(label="Mensaje"),
11         gr.Slider(0, 100, label="Puntuación"),
12         gr.Image(label="Sube una foto")
13     ],
14     outputs=["text", "number", "image"]
15 )
16
17 demo.launch()
```

Bloques (Blocks) para Layouts Complejos

Si necesitas más control que el que ofrece `Interface`, usa `gr.Blocks`.

```
1  import gradio as gr
2
3  with gr.Blocks() as demo:
4      gr.Markdown("# Sistema de Análisis")
5
6      with gr.Row():
7          inp = gr.Textbox(placeholder="Escribe algo aquí...")
8          out = gr.Textbox()
9
10     btn = gr.Button("Ejecutar")
11     btn.click(fn=lambda x: x.upper(), inputs=inp, outputs=out)
12
13  demo.launch()
```

Compartiendo tu interfaz

Para crear un enlace público temporal (válido por 72 horas), simplemente añade `share=True` al método `launch()`.

```
1 demo.launch(share=True)
```

Esto te proporcionará una URL tipo `https://12345.gradio.app` que puedes enviar a cualquier persona para que pruebe tu demo directamente desde tu máquina.

Integración con Hugging Face Spaces

Gradio se integra de forma nativa con **Hugging Face Spaces**, permitiéndote alojar tus demos de forma gratuita y permanente.

- **Fácil despliegue:** Sube un archivo `app.py` y un `requirements.txt`.
- **Visibilidad:** Haz que tu modelo sea accesible para toda la comunidad.
- **Hardware:** Opciones para usar GPUs gratuitas o de pago.

```
1  # Puedes cargar modelos directamente desde el Hub
2  import gradio as gr
3
4  gr.Interface.load("huggingface/gpt2").launch()
```


Ejemplos de Elementos Avanzados

Gradio incluye componentes para manejar casi cualquier tipo de dato:

Componente	Uso
<code>gr.Audio</code>	Grabar o subir archivos de audio
<code>gr.Video</code>	Reproducir o capturar vídeo
<code>gr.File</code>	Subir/Bajar archivos arbitrarios
<code>gr.Dataframe</code>	Visualizar y editar tablas tipo Excel
<code>gr.Chatbot</code>	Interfaz optimizada para LLMs

Ejemplo: Interfaz de Chat con Ollama

Integrar Ollama es sencillo usando su librería oficial. Esto permite conectar tu interfaz con modelos locales como Llama 3 o Mistral.

```
1  import gradio as gr
2  import ollama
3
4  def responder(mensaje, historia):
5      response = ollama.chat(
6          model="llama3.2",
7          messages=[{'role': 'user', 'content': mensaje}],
8      )
9      return response['message']['content']
10
11 demo = gr.ChatInterface(fn=responder, title="Chat Local con Ollama")
12 demo.launch()
```

Chat con Ollama y Streaming

Para una experiencia fluida, podemos usar el modo `stream=True` de Ollama junto con `yield` en Gradio.

```
1  import gradio as gr
2  import ollama
3
4  def responder_streaming(mensaje, historia):
5      response = ollama.chat(
6          model="llama3",
7          messages=[{'role': 'user', 'content': mensaje}],
8          stream=True
9      )
10
11     mensaje_completo = ""
12     for chunk in response:
13         mensaje_completo += chunk['message']['content']
14         yield mensaje_completo
15
16 gr.ChatInterface(fn=responder_streaming).launch()
```

Personalización del Chat (UI Atractiva)

Puedes añadir ejemplos predefinidos, cambiar el tema y añadir un icono para hacerlo más visual.

```
1  import gradio as gr
2
3  demo = gr.ChatInterface(
4      fn=responder,
5      title="👑 IA Avanzada",
6      description="Pregunta lo que quieras a tu asistente personal.",
7      examples=["¿Cómo funciona Gradio?", "¿Cómo conecto Ollama?"],
8      theme="soft", # Temas: 'soft', 'glass', 'monochrome', etc.
9      cache_examples=True,
10     retry_btn=None, # Personaliza o quita botones
11     undo_btn="Deshacer",
12     clear_btn="Limpiar",
13 )
14
15 demo.launch()
```

Streaming: Respuestas en Tiempo Real

Para una experiencia tipo ChatGPT, usa generadores (`yield`) para mostrar la respuesta palabra a palabra.

```
1  import gradio as gr
2  import time
3
4  def responder_streaming(mensaje, historia):
5      respuesta = f"Generando una respuesta larga para: {mensaje}..."
6      for i in range(len(respuesta)):
7          time.sleep(0.05)
8          yield respuesta[:i+1]
9
10 gr.ChatInterface(fn=responder_streaming).launch()
```

¡Gracias!

¿Listo para crear tu primera interfaz?

Documentación oficial